

Función en la nube bajo modelo PaaS

API REST en Python (Flask) desplegada en Render

Universidad Antonio Nariño (UAN) - Sede Sur, Bogotá D.C.

Gestión de Proyectos de Software - 2026-II

Docente: Ing. Jhonatan Sneider Rico Pinto

Integrante(s): _____

Fecha: 28 de septiembre de 2026

URL del servicio desplegado: _____

1. Objetivo y plataforma elegida

Se implementó una pequeña API REST que recibe un número entero n y devuelve su factorial, si es primo y su valor de Fibonacci. Se eligió **Render** (render.com) por ser una plataforma PaaS: se entrega el código en un repositorio Git y la plataforma se encarga de construir, desplegar, exponer la URL con HTTPS y mantener el servidor, sin administrar máquinas virtuales. Además, su nivel gratuito no exige tarjeta de crédito para el servicio web básico [1].

Cómo se usa: GET /analizar/<n> (n entre 0 y 500) y GET / para el estado del servicio.

2. Código fuente

app.py

```
"""API REST de utilidades numericas - desplegable en Render (PaaS).

Endpoints:
    GET /                -> estado del servicio
    GET /analizar/<n>    -> factorial, primalidad y Fibonacci de n
"""
from flask import Flask, jsonify

app = Flask(__name__)

MAX_N = 500 # limite para evitar consumo excesivo de CPU en el plan gratuito


def factorial(n: int) -> int:
    resultado = 1
    for i in range(2, n + 1):
        resultado *= i
    return resultado


def es_primo(n: int) -> bool:
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True


def fibonacci(n: int) -> int:
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a


@app.route("/")
def inicio():
    return jsonify(servicio="API de utilidades numericas", estado="ok",
                  uso="/analizar/<n> con n entero entre 0 y %d" % MAX_N)


@app.route("/analizar/<int:n>")
def analizar(n):
    if n < 0 or n > MAX_N:
        return jsonify(error="n debe estar entre 0 y %d" % MAX_N), 400
    return jsonify(n=n, factorial=str(factorial(n)),
                  es_primo=es_primo(n), fibonacci=str(fibonacci(n)))
```

```
@app.errorhandler(404)
def no_encontrado(_):
    return jsonify(error="ruta no encontrada; use /analizar/<n>"), 404

if __name__ == "__main__":
    import os
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT", 5000)))
```

requirements.txt y Procfile

```
# requirements.txt
flask==3.1.3
gunicorn==23.0.0

# Procfile (comando de inicio)
web: gunicorn app:app
```

En Render, el *Build Command* es `pip install -r requirements.txt` y el *Start Command* es `gunicorn app:app`.

test_app.py (pruebas automáticas)

```
import unittest
from app import app, factorial, es_primo, fibonacci

class PruebasFunciones(unittest.TestCase):
    def test_factorial(self):
        self.assertEqual(factorial(0), 1)
        self.assertEqual(factorial(5), 120)
        self.assertEqual(factorial(10), 3628800)

    def test_es_primo(self):
        self.assertFalse(es_primo(0))
        self.assertFalse(es_primo(1))
        self.assertTrue(es_primo(2))
        self.assertTrue(es_primo(97))
        self.assertFalse(es_primo(100))

    def test_fibonacci(self):
        self.assertEqual(fibonacci(0), 0)
        self.assertEqual(fibonacci(1), 1)
        self.assertEqual(fibonacci(10), 55)

class PruebasAPI(unittest.TestCase):
    def setUp(self):
        self.c = app.test_client()

    def test_inicio(self):
        r = self.c.get("/")
        self.assertEqual(r.status_code, 200)
        self.assertEqual(r.get_json()["estado"], "ok")

    def test_analizar_valido(self):
        r = self.c.get("/analizar/7")
        self.assertEqual(r.status_code, 200)
        d = r.get_json()
        self.assertEqual(d["factorial"], "5040")
        self.assertTrue(d["es_primo"])
        self.assertEqual(d["fibonacci"], "13")

    def test_fuera_de_rango(self):
        self.assertEqual(self.c.get("/analizar/501").status_code, 400)
        self.assertEqual(self.c.get("/analizar/-3").status_code, 404)

    def test_ruta_inexistente(self):
        self.assertEqual(self.c.get("/otra").status_code, 404)

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

3. Pasos de despliegue en Render

1) Subir la carpeta (app.py, test_app.py, requirements.txt, Procfile) a un repositorio de GitHub. 2) En render.com crear cuenta y elegir **New > Web Service**, conectando el repositorio. 3) Runtime: Python 3; Build Command: `pip install -r requirements.txt`; Start Command: `gunicorn app:app`; Instance Type: Free. 4) Pulsar *Deploy* y esperar el estado *Live*; la plataforma entrega una URL pública del tipo `https://nombre.onrender.com`.

4.3 Evidencias del despliegue en la nube (Render)

Captura del panel de Render con el servicio en estado **Live** y su URL pública.

PEGAR AQUÍ: captura del dashboard de Render (servicio 'Live')

Captura del navegador en `https://SU-SERVICIO0.onrender.com/analizar/7`.

PEGAR AQUÍ: captura del navegador con la respuesta JSON de `/analizar/7`

Captura de un caso de error (por ejemplo `/analizar/501`, respuesta 400).

PEGAR AQUÍ: captura de `/analizar/501` (error 400)

5. Resumen de la plataforma: Render

5.1 Funcionalidades principales

Render es una plataforma en la nube que permite construir, desplegar y escalar aplicaciones sin gestionar infraestructura, mediante despliegues desde Git, bases de datos administradas y soporte nativo de Docker [5]. Ofrece servicios web, sitios estáticos, workers en segundo plano, cron jobs, PostgreSQL y Redis (Key Value), con TLS gratuito, despliegues sin caída e infraestructura como código (Blueprints) [2][5].

5.2 ¿Nube comercial o infraestructura propia?

Render opera como una capa PaaS sobre un proveedor de nube comercial: su página de estado reporta incidentes causados por su proveedor upstream AWS (por ejemplo, en la región de Oregon) [3]. Es decir, no posee centros de datos propios para este servicio, sino que abstrae la infraestructura de AWS.

5.3 Aspectos relevantes

Despliegue simple, sin Dockerfile ni CLI obligatorios [1]; instancia gratuita de 512 MB de RAM y 0,1 CPU, suficiente para aplicaciones ligeras [2]; los servicios gratuitos se suspenden tras 15 minutos sin tráfico y tardan cerca de un minuto en despertar, por lo que sirven para demos y prototipos, no para producción [2][6]; las bases de datos PostgreSQL gratuitas expiran a los 30 días de su creación [6].

5.4 Precios de uso (referenciales, USD)

Concepto	Precio
Workspace Hobby	\$0 / mes
Workspace Pro (por workspace, no por usuario)	\$25 / mes [4]
Workspace Scale	\$499 / mes [4]
Servicio web Free (512 MB, 0,1 CPU)	\$0 (750 h de instancia/mes) [1][6]
Servicio web Starter	\$7 / mes [2]
Servicio web Standard	\$25 / mes [2]
Ancho de banda adicional	\$0,15 por GB [4]
PostgreSQL de pago	desde \$6 / mes [7]

Render adoptó su modelo actual de planes por workspace el 23 de abril de 2026 [4]. Los precios cambian con frecuencia; conviene verificarlos en render.com/pricing antes de decidir. También se han reportado cobros inesperados en cuentas gratuitas por exceso de ancho de banda [7].

5.5 Disponibilidad

Las instancias gratuitas se suspenden por inactividad y se apagan al agotar las 750 horas mensuales hasta el mes siguiente [1]. Los niveles de producción de pago se describen como respaldados por SLA y con patrones de redundancia regional [8]. Render publica el estado y el historial de 90 días en status.render.com [3]; el 27 de septiembre de 2026 se registró un incidente por inestabilidad del proveedor AWS en Oregon, en el cual todos los servicios se recuperaron salvo, durante un tiempo, el nivel gratuito [3].

6. Referencias

- [1] Render, «Platforms with a real free tier for developers in 2026». render.com/articles/platforms-with-a-real-free-tier-for-developers-in-2026
- [2] SRVRLSS, «Render Pricing 2026: Free Tier, RAM Limits & Alternatives». srvrlss.io/provider/render
- [3] Render Status. status.render.com (consultado el 28/09/2026)
- [4] Kuberns, «Render Pricing Guide» (14/08/2026). kuberns.com/blogs/render-pricing
- [5] CheckThat.ai, «Render Pricing 2026: Total Cost & Competitors Compared» (30/07/2026). checkthat.ai/brands/render/pricing
- [6] LiveMy.app, «Render pricing explained» (10/08/2026). livemy.app/blog/render-pricing
- [7] CheckThat.ai (ibid.) para PostgreSQL y cobros en cuentas gratuitas.
- [8] RFP.wiki, «Render - Reliability: Uptime, SLAs, Incidents (2026)». rfp.wiki/cloud-computing/cloud-native-application-platforms/render